FITNESS DUAL

Project Scope & Technical Plan

Frontend	React Native mobile application
Backend	Flask REST API + database
AI Service	FastAPI service for pose detection, exercise analysis and repetition counting

This document outlines the proposed screens, Flask backend modules, FastAPI endpoints, and database structure. The final implementation should be adjusted to the existing codebase and the agreed exercise list.

1. Frontend — Proposed 20 Screens

Group	Proposed screens
Account & onboarding (3)	1. Splash screen / app logo 2. Login screen 3. Sign-up screen
Main app (5)	4. Home dashboard 5. Workout categories 6. Exercise details 7. Manual workout 8. AI workout (live camera)
Fitness tracking (5)	9. Progress dashboard / charts 10. Workout history 11. Nutrition dashboard 12. Meal plans / recipes 13. Water intake tracking
User account (4)	14. User profile 15. Edit profile 16. Fitness goals 17. Settings
Additional screens (3)	18. Workout summary / results 19. Notifications 20. Help & support

Note: These are proposed screens, not a requirement to build 20 entirely separate layouts. Shared navigation, cards, headers, dialogs, and other reusable components can be used across screens.

2. Flask Backend — Proposed API Modules

Module	Example operations	Est.
Authentication	Register, login, logout, refresh token	4
User profile	Get profile, update profile, change password	3
Exercises	Exercise list, categories, exercise details	3
Workouts	Create, start, complete, workout history	4

Module	Example operations	Est.
Fitness progress	Dashboard, statistics, weekly progress	3
Nutrition	Meal plans, meals, food logs	4
Water intake	Add water, daily water summary	2
Fitness goals	Set, view and update goals	3
Notifications	List notifications, mark as read	2
Settings	Get and update settings	2
Total (planning estimate)	Actual count may vary by API design.	**30**

3. FastAPI — AI Service Endpoints

FastAPI is proposed as a separate AI service. Flask can call it when AI processing is needed; user accounts and persistent app data remain in the Flask backend/database.

A. Pose detection

• POST /ai/pose/detect — detect pose/keypoints from an input frame or image.

• POST /ai/pose/analyze — analyze detected keypoints and return structured pose information.

B. Repetition counting

• POST /ai/reps/start — initialize a workout/repetition-counting session.

• POST /ai/reps/update — update count/state using new pose data.

• POST /ai/reps/finish — return session totals and finish the counting session.

C. Exercise analysis

• POST /ai/exercise/classify — classify among supported exercises, if a suitable model is available.

• POST /ai/posture/feedback — provide basic posture feedback from supported rules/model outputs.

D. Workout analysis and service health

• POST /ai/workout/analyze — summarize supported workout-session data.

• GET /ai/health — report whether the AI service is available.

FastAPI planning estimate: 9 endpoints. The exact design depends on the chosen pose model and whether camera processing runs on-device, through HTTP requests, or via a streaming connection. Sending every video frame as an individual HTTP request is usually not the best approach.

4. Proposed Database Tables

Table	Purpose
users	Account and profile information
exercises	Exercise names, instructions and categories
workouts	Workout sessions and session status
workout_exercises	Exercise sets, reps and duration within sessions
progress_records	Historical fitness performance
fitness_goals	User goals and goal status
meal_plans / food_logs	Nutrition plans and logged food

Table	Purpose
water_logs	Water intake entries
notifications	User notifications and read status
user_settings	User preferences

5. Implementation Notes

• Use the existing React Native frontend where possible; first review the current code before replacing screens.

• Agree on the supported exercise list, pose-feedback rules, and expected accuracy before development.

• Reuse existing pretrained models where suitable. Training new models from scratch is outside this assumed scope unless separately agreed.

• Hosting, paid APIs, third-party subscriptions, and deployment fees are not included in this scope document.

• The screen and endpoint counts are planning estimates; combine or split endpoints based on implementation needs.